KU-SBB qualification report — team BreakPoint

Designated qualification submission: **submission_blendG2.csv** (Kaggle public AUC **0.86386**, out-of-fold 0.8657; GPU route, see §3 and §6). CPU-auditable fallback, also selected on Kaggle: submission_blendD_r12.csv (public 0.86076, OOF 0.8606), reproduced bit-for-bit by `predict.py` in 31 s on 2 cores.

1. Task and validation design

Each series is a univariate sequence with a known boundary (period 0 / 1); the target is whether a lasting process intervention happened at the boundary (ROC-AUC). All numbers below are out-of-fold (OOF) over **StratifiedKFold(5, shuffle, seed 42) on whole series** (24,000 training series; no series is split). Everything learned from data is fitted inside the training part of a fold: early stopping uses an inner 10% split of the fold-train rows; feature selection for interaction features is redone in every fold on the inner-train rows only; pseudo-label and label-cleaning experiments were nested. Differences between variants are judged with a paired bootstrap of the OOF AUC (95% CI) and by per-fold agreement, never by the public leaderboard (public AUC on ~1,520 rows has a standard deviation of about 0.009, larger than most effects we test).

2. Features (all per series, computed independently of labels)

- **Classical change statistics** (`features.py`, `harvest.py`): pre/post moments, quantiles, variance ratios, Levene/KS/Mood/AD/energy tests, CUSUM/Chow-type statistics, autocorrelation and spectral changes, boundary windows of several widths. Ideas adapted from public write-ups of the 2025 ADIA structural-break challenge (no code copied; see README).
- **Residual transforms** (`factory.py`): the same statistics on AR(10) prediction residuals and their absolute values fitted on the pre segment — the largest single gain of the project (OOF 0.833 → 0.841).
- **Generator-informed likelihood features**: Gaussian likelihood-ratio tests for level / spread / dependence / noise-type changes with a drift-aware null (`probe/joint_lr.py`, `probe/noise_lr.py`, `probe/gen_feats.py`, `probe/model_feats.py`).
- **Semi-automatic simulation features** (team-generated synthetic data, allowed by the rules; generator code kept in `probe/ku_gen.py`, `probe/sim_series_ku2.py`): we built a synthetic generator calibrated to the training data (families WN/AR/SIN/RW/local-level, t/GARCH noise, level/spread/dependence/shape breaks, nuisances), simulated 30k labelled fakes per bag, and trained LightGBM estimators that predict the hidden generator parameters (family, break kind, break sizes, noise type, ...) from the same feature set. Their predictions on real series form 31 columns, averaged over 6 independent fake bags (optionally realism-weighted with a real-vs-fake discriminator).
- **Time-reversal augmentation**: a structural change at the boundary is still a change when the series is read backwards, so every series is reversed (boundary at $n - n_0$, same label), all feature families are recomputed on the reversed series, models are trained on originals + reversed copies and predict the mean of both (test-time augmentation). Single pair arm 0.8575 → 0.8599 OOF (+0.0025, CI [+0.0009, +0.0041]). Negation and cropping were tested the same way and did not hold up.
- **Pairwise rank interactions** (`probe/confirm_pair.py`): in each fold, the top-K features by gain of a quick model fitted on the fold's inner-train rows are rank-transformed and all pairwise differences and products are added (K=60 gives 3,540 columns). This lifted the best single model from 0.8559 to 0.8575 OOF.

3. Models and the designated blend

All models are LightGBM (lr 0.02, 31 leaves, min_child 40, feature_fraction 0.5, bagging 0.8, early stopping 300 on the

inner split, 3 seeds per fold). The designated submission is a fixed-weight rank blend:

blendG2 = rank(2·pair arms + 1·interaction arm + 2·time-reversal pair arms + 6·TabPFN arms + 6·reversed-context TabPFN arms) (each group a mean of ranks). The TabPFN arms run the public pretrained TabPFN 3.5 model on each fold's top-K features (ranks, K 60–100, whole fold-train as context); the reversed-context arms put reversed copies into the context. The CPU fallback blendD uses only the first two groups.

The 11 pair arms differ in K (40–80), the seed of the per-fold feature selection and the semi-automatic feature bag; the interaction arm takes the top-162 pool features, builds ~91k pairwise operations (product, sum, difference, ratios, $a \cdot b^2$, squares) inside each fold, and retrains on the 800 highest-gain columns. Blend weights were chosen by cross-validation over a small grid (weights picked on 4 folds, scored on the 5th) and then frozen; no weight was tuned on the public leaderboard.

Model / blend	**OOF AUC**	**Public AUC**
LightGBM, 242 classical features (baseline pipeline)	0.8327	–
+ residual transforms, generator LR features (combo10gnj)	0.8539	0.85415
+ semi-automatic simulation features (single LGB, bag6w)	0.8559	–
Single pair-interaction arm (K=60)	0.8575	–
Earlier best blend (saf + bag6 + 2 interaction arms)	0.8582	0.85830
blendD / blendD_r12 (CPU fallback): 11 pair arms ×2 + interaction ×1	0.8606	0.86076
+ 8 TabPFN arms (blendF)	0.8641	0.86324
blendG2 (designated): + time-reversal pair arms + reversed-context TabPFN	**0.8657**	**0.86386**

Each step is a paired-bootstrap improvement higher in all 5 folds: blendD +0.0018 vs earlier best; blendG2 +0.0016 [+0.0010, +0.0022] vs blendF.

4. Error and robustness analysis

- **By series family** (families estimated by the semi-automatic models): random-walk series are the weak spot (AUC ~0.70 vs 0.83–0.89 elsewhere, false-alarm rate ~0.5). An oracle on simulated RW series that knows the true parameters reaches only ~0.76, so most of this gap is inherent. Shape (noise-distribution) breaks are the hardest break kind (0.78). Short segments (≤160 points) score 0.81 vs 0.89 for long ones.
- **Train/test shift**: adversarial validation separates train from test at AUC 0.59. The driver is a group of test series whose low-order floating-point bits look rescaled (17.9% of test vs 1.7% of train); these series are almost white noise. Our predictions on them are less extreme. Importance re-weighting towards the test distribution hurt OOF (−0.0047), and fold-clean pseudo-labelling gave only +0.0009 for one arm, so neither is in the designated blend.
- **Ceiling**: on our simulator, a likelihood-ratio oracle with the true parameters reaches ~0.91, and with parameters estimated from the series ~0.86–0.87; the remaining headroom is mostly in spread, dependence and shape breaks.
- **What did not help** (all checked with CIs): XGBoost/CatBoost/ExtraTrees/MLP/kNN/logistic arms on the same features, raw-sequence CNNs (0.75–0.83), MiniRocket-style kernels, label-noise down-weighting, per-family specialist models (their apparent gain equalled a two-seed control), and dozens of extra feature families (MOSUM, ordinal patterns, MMD, BDS/Hurst/ADF, change-point scans).

5. Limitations

- The synthetic generator is a reverse-engineered approximation (real-vs-fake classifier AUC ~0.60 on no-break and ~0.67 on break series); the semi-automatic features inherit its biases.
- Many variants were compared on the same OOF folds; reported gains are paired-bootstrap CIs but not corrected for multiple comparisons, and quick-screen gains typically halved at production settings.
- **Resources.** The best model needs a GPU: TabPFN inference did not finish 100 test rows in 7 minutes on 2 CPU cores, and distilling it into LightGBM lost the gain. Our designated GPU route therefore asks for one consumer GPU (≥16 GB) for the audit (the whole blend runs in minutes); with more compute we expect further gains from more TabPFN/time-reversal bags. Within the CPU-only budget the fully auditable fallback blendD_r12 is provided and selected.
- The full training pipeline (feature generation, 6 synthetic bags, interaction arm) takes several hours on a multi-core HPC node; the audit route is the frozen checkpoints plus the documented training commands (TRAINING.md).

6. Reproducibility, hardware, sources and AI use

Inference: designated GPU route — TRAINING_GPU.md + `make_blendG2.py`; CPU fallback — `python predict.py --data data --feat outputs --ckpt ckpt --out submission.csv --threads 2` (bit-exact, 31 s). **Training hardware**: KU Madar HPC CPU nodes (64–96 cores, Slurm), a local 32-core workstation; two local RTX 4090 GPUs for the TabPFN arms of the designated submission (and exploratory CNNs). Python 3.12/3.13, LightGBM 4.6.0, scikit-learn 1.7, NumPy 1.26, pandas 2.3, SciPy 1.16 (requirements.txt). Randomness: every split, model and generator seed is fixed in code. **Data**: only the supplied competition data plus team-generated synthetic series (generator code included); no external datasets. **Sources**: public ADIA 2025 write-ups (ideas only), Page (1954) CUSUM, Andrews (1993), Dempster et al. (2020) ROCKET, TabPFN 3.5 (Hollmann et al., public pretrained weights). **AI use**: an AI coding assistant (Anthropic Claude, via Claude Code) helped write and run analysis code, design experiments and draft this report under continuous human review; the assistant executed code on our own machines; its context contained summary statistics, scores and occasionally a few printed values of individual series while debugging, but no competition files, label files or prediction files were uploaded to any third-party AI service (data stayed on the local workstation and the university HPC). Details in AI_DISCLOSURE.md.